Chương 4

MÔ HÌNH MẠNG HỒI QUY RNN

Trong các loại dữ liệu thực tế, có một dữ liệu đặc biệt đó là dữ liệu tuần tự theo dòng thời gian, ví dụ như dữ liệu text, tiếng nói. Từ đó, trong các mô hình học sâu cũng đưa ra mô hình có khả năng xử lý dữ liệu tuần tự hoặc thời gian, nơi mà ngữ cảnh và thứ tự đóng vai trò quan trọng. Mô hình RNN - Recurrent Neural Networks (Mạng nơ-ron hồi quy) được thiết kế để giải quyết chính nhu cầu này. Khác với các mạng nơ-ron truyền thống chỉ hoạt động trên dữ liệu độc lập và không quan tâm đến các mối quan hệ theo thời gian, RNN có khả năng ghi nhớ trạng thái trước đó, cho phép chúng học được các mẫu và mối liên hệ tuần tự trong dữ liệu.

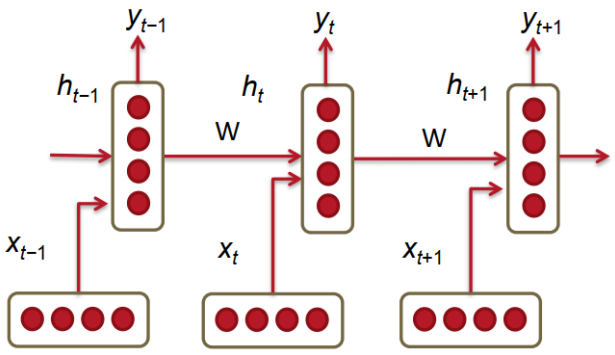
Từ nhận diện giọng nói, xử lý ngôn ngữ tự nhiên, dịch máy, đến dự đoán chuỗi thời gian, RNN đã trở thành một trong những công cụ mạnh mẽ nhất để giải quyết các bài toán phức tạp liên quan đến dữ liệu tuần tự.

Chương này cung cấp các nội dung sau:

- Kiến thức nền tảng về RNN, bao gồm cách thức hoạt động, các thách thức của RNN.
- Các mô hình cải tiến của RNN như LSTM và GRU.
- Các biến thể của LSTM
- Ứng dụng của RNN/LSTM trong một số các bài toán phổ biến như các bài toán cơ bản trong xử lý ngôn ngữ tự nhiên - NLP.
- Giới thiệu về mô hình Transformer. Đây là nền tảng cho các mô hình ngôn ngữ lớn sau này.

4.1 Mô hình mạng RNN

Không giống như các mô hình dịch truyền thống, nơi chỉ một cửa sổ giới hạn các từ trước đó được sử dụng để xây dựng ngữ cảnh trong mô hình ngôn ngữ, Mạng RNN có khả năng xây dựng ngữ cảnh dựa trên toàn bộ các từ trước đó trong tập dữ liệu.



Hình 4.1: Mô hình RNN

Hình 4.1 minh họa kiến trúc của RNN, trong đó mỗi hộp chữ nhật đại diện cho một lớp ẩn tại một bước thời gian t. Mỗi lớp ẩn này bao gồm một số lượng nhất định các nơ-ron, trong đó mỗi nơ-ron thực

hiện một phép toán ma trận tuyến tính trên các đầu vào, sau đó là một phép toán phi tuyến (ví dụ: hàm kích hoạt tanh()). Tại mỗi bước thời gian, lớp ẩn nhận hai loại đầu vào:

- Đầu ra từ lớp ẩn tại bước thời gian trước đó, h_{t-1}.
- Đầu vào tại bước thời gian hiện tại x_t.

Đầu ra h_t được tính qua đầu vào thứ nhất h_{t-1} được nhân với một ma trận trọng số W^{hh} và đầu vào thứ hai x_t được nhân với một ma trận trọng số W^{hx} (Công thức 4.1)

$$h_t = \sigma\left(W^{(hh)}h_{t-1} + W^{(hx)}x_{[t]}\right). \tag{4.1}$$

Sau đó, đầu ra h_t được nhân tiếp với một ma trận trọng số khác W^S và được đưa qua một lớp softmax để dự đoán đầu ra tương ứng với từ tiếp theo trong từ vựng (Công thức 4.2).

$$\hat{y}_t = \text{softmax}\left(W^{(S)}h_t\right) \tag{4.2}$$

Các đầu vào và đầu ra của từng nơ-ron trong mô hình được minh họa chi tiết trong Hình 4.1.

Điều thú vị ở đây là các trọng số $W^{(hh)}$ và $W^{(hx)}$ giống nhau được áp dụng lặp đi lặp lại tại mỗi bước thời gian. Nhờ vậy, số lượng tham số mà mô hình cần phải học ít hơn và quan trọng nhất, nó không phụ thuộc vào độ dài của chuỗi đầu vào. Dưới đây là các chi tiết liên quan đến từng tham số trong mô hình RNN:

1. $x_1, \ldots, x_{t-1}, x_t, x_{t+1}, \ldots, x_T$: là các vector từ, tương ứng với một tập dữ liệu chứa T từ.
2. $h_t = \sigma\left(W^{(hh)}h_{t-1} + W^{(hx)}x_t\right)$: mối quan hệ được sử dụng để tính toán các đặc trưng đầu ra của lớp ẩn tại mỗi bước thời gian t. Trong đó:
   - $x_t \in \mathbb{R}^d$: vector từ đầu vào tại bước thời gian t.
   - $W^{(hx)} \in \mathbb{R}^{D_h \times d}$: ma trận trọng số được sử dụng để xử lý vector từ đầu vào x_t.
   - $W^{(hh)} \in \mathbb{R}^{D_h \times D_h}$: ma trận trọng số được sử dụng để xử lý đầu ra từ bước thời gian trước đó, h_{t-1}.
   - $h_{t-1} \in \mathbb{R}^{D_h}$: đầu ra của hàm phi tuyến tại bước thời gian trước đó, $t-1$. Đặc biệt, $h_0 \in \mathbb{R}^{D_h}$ là vector khởi tạo cho lớp ẩn tại bước thời gian $t = 0$.
   - $\sigma()$: hàm phi tuyến, trong trường hợp này là hàm sigmoid.
3. $\hat{y}_t = \text{softmax}(W^{(S)}h_t)$: phân phối xác suất đầu ra trên tập từ vựng tại mỗi bước thời gian t. Về bản chất, $\hat{y}_t$ là từ được dự đoán tiếp theo dựa trên điểm ngữ cảnh của tài liệu đã tính đến cho đến thời điểm đó (tức là h_{t-1}) và vector từ cuối cùng quan sát được $x^{(t)}$. Ở đây, $W^{(S)} \in \mathbb{R}^{|V| \times D_h}$ và $\hat{y}_t \in \mathbb{R}^{|V|}$, với $|V|$ là kích thước của tập từ vựng.

Các tham số này tạo thành nền tảng cho việc tính toán và học trong mô hình RNN, giúp nó nắm bắt được các mối quan hệ tuần tự trong dữ liệu.

Một ví dụ về mô hình ngôn ngữ RNN được minh họa trong Hình 4.2. Cách ký hiệu trong hình này hơi khác biệt: ở đây, $W^{(hh)}$ tương ứng với W_h, $W^{(hx)}$ tương ứng với W_e, và $W^{(S)}$ tương ứng với U. Ma trận E chuyển đổi các từ đầu vào $x^{(t)}$ thành các vector nhúng từ - word embedding $e^{(t)}$. Lớp softmax cuối cùng trên tập từ vựng cho thấy xác suất của các lựa chọn khác nhau cho token $x^{(5)}$, được điều kiện hóa bởi tất cả các token trước đó. Dữ liệu đầu vào có thể dài hơn nhiều so với 4-5 token.

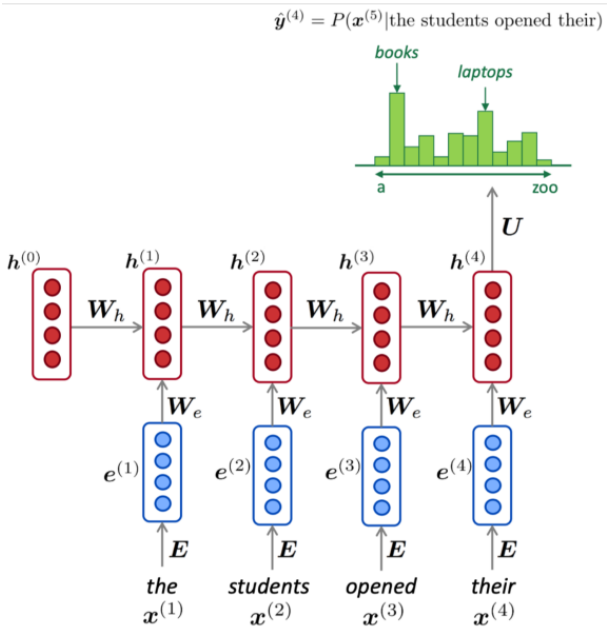
4.1.1 Hàm mất mát

Hàm mất mát được sử dụng trong RNN thường là hàm sai số cross-entropy. Công thức 4.7 mô tả hàm này dưới dạng tổng trên toàn bộ tập từ vựng tại bước thời gian t.

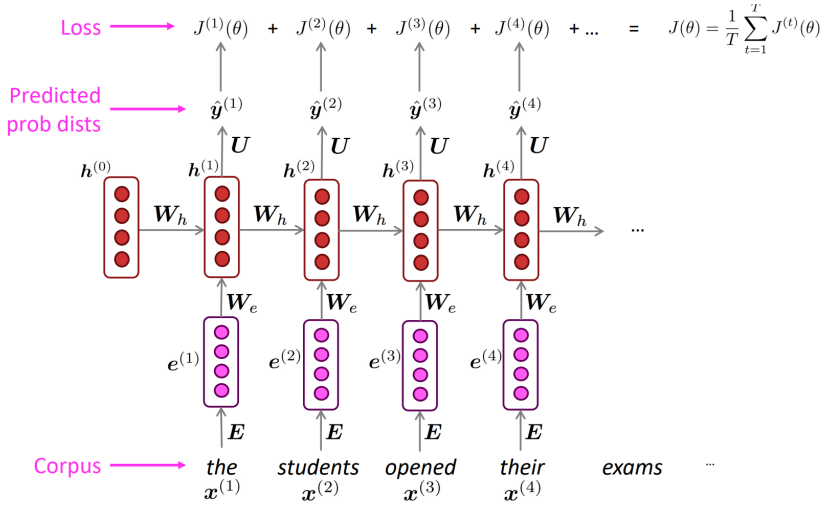
$$J^{(t)}(\theta) = -\sum_{j=1}^{|V|} y_{t,j} \times \log(\hat{y}_{t,j}) \tag{4.3}$$

Hàm mất mát cross-entropy trên một tập văn bản có kích thước T được định nghĩa (Hình 4.3)như sau:

$$J = \frac{1}{T}\sum_{t=1}^{T} J^{(t)}(\theta) = -\frac{1}{T}\sum_{t=1}^{T}\sum_{j=1}^{|V|} y_{t,j} \times \log(\hat{y}_{t,j}) \tag{4.4}$$



Hình 4.2: Mô hình ngôn ngữ RNN cho bài toán dự đoán từ tiếp theo trong câu



Hình 4.3: Hàm mất mát trong huấn luyện mô hình RNN

4.1.2 Ưu nhược điểm của RNN

RNN có một số ưu điểm:

- Chúng có thể xử lý các chuỗi đầu vào có độ dài bất kỳ.
- Kích thước của mô hình không tăng lên đối với các chuỗi đầu vào dài hơn.
- Việc tính toán tại bước t (về lý thuyết) có thể sử dụng thông tin từ nhiều bước trước đó.
- Các trọng số giống nhau được áp dụng cho mọi bước thời gian của đầu vào, tạo ra sự đối xứng trong cách xử lý các đầu vào.

Tuy nhiên, RNN cũng có một số nhược điểm:

- Việc tính toán chậm - do quá trình xử lý mang tính tuần tự, không thể song song hóa.

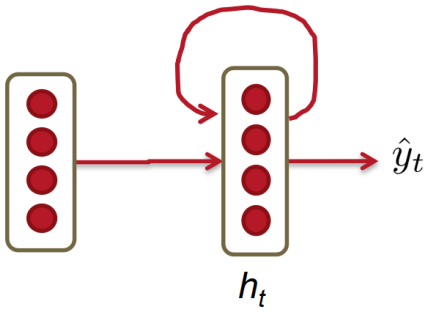
- Trong thực tế, rất khó để truy cập thông tin từ nhiều bước trước đó do các vấn đề như tiêu biến gradient (vanishing gradient) và bùng nổ gradient (exploding gradient), những vấn đề sẽ được thảo luận trong phần tiếp theo.

Lượng bộ nhớ cần thiết để chạy một tầng của RNN tỷ lệ thuận với số lượng từ trong tập dữ liệu. Ta có thể xem một câu như một minibatch, và một câu chứa k từ sẽ cần lưu trữ k vector từ trong bộ nhớ. Ngoài ra, RNN phải duy trì hai cặp ma trận trọng số W và các bias b. Như đã đề cập, mặc dù kích thước của W có thể rất lớn, nhưng nó không tăng lên theo kích thước của tập dữ liệu (khác với các mô hình ngôn ngữ truyền thống).

Ví dụ, với một RNN có 1000 tầng tái phát, ma trận sẽ có kích thước 1000×1000 bất kể kích thước của tập dữ liệu.

RNN có thể được sử dụng cho nhiều tác vụ khác nhau, chẳng hạn như gán nhãn (ví dụ: nhận dạng từ loại, nhận dạng thực thể có tên), phân loại câu (ví dụ: phân tích cảm xúc), và các module mã hóa (ví dụ: trả lời câu hỏi, dịch máy và nhiều tác vụ khác). Trong hai ứng dụng sau, chúng ta cần một biểu diễn cho toàn bộ câu, mà có thể được tạo ra bằng cách tính giá trị lớn nhất hoặc giá trị trung bình theo từng phần tử của tất cả các trạng thái ẩn tại các bước thời gian trong câu đó.

Lưu ý: Hình 4.4 là một biểu diễn thay thế của RNN được sử dụng trong một số tài liệu. Biểu diễn này thể hiện lớp ẩn của RNN như một vòng lặp.



Hình 4.4: Mô hình RNN lặp qua các bước thời gian t

4.1.3 Vấn đề tiêu biến và bùng nổ đạo hàm

RNN là mô hình mạnh mẽ để xử lý dữ liệu tuần tự, nhưng chúng thường gặp phải các vấn đề liên quan đến gradient trong quá trình huấn luyện, đặc biệt là khi làm việc với các chuỗi dữ liệu dài. Hai vấn đề phổ biến là tiêu biến gradient (vanishing gradient) và bùng nổ gradient (exploding gradient).

1. Tiêu biến Gradient (Vanishing Gradient)

Tiêu biến đạo hàm là hiện tượng khi huấn luyện RNN, gradient của hàm mất mát (loss) được lan truyền ngược qua các bước thời gian để cập nhật các trọng số. Trong quá trình này, gradient có thể trở nên rất nhỏ khi lan truyền ngược qua nhiều bước thời gian. Điều này xảy ra do việc nhân liên tiếp các giá trị nhỏ (liên quan đến ma trận trọng số và đạo hàm của các hàm kích hoạt như sigmoid hoặc tanh) dẫn đến gradient bị thu nhỏ về gần 0.

Từ hiện tượng trên dẫn tới hậu quả là các gradient quá nhỏ làm cho việc cập nhật trọng số trở nên không đáng kể, dẫn đến mô hình không học được mối quan hệ ở các bước thời gian xa. Do đó, mô hình chỉ có thể học từ các bước thời gian gần, làm mất đi khả năng ghi nhớ dài hạn.

Giải pháp:

- Sử dụng các biến thể của RNN như LSTM (Long Short-Term Memory) và GRU (Gated Recurrent Unit), được thiết kế để khắc phục vấn đề tiêu biến gradient bằng cách kiểm soát dòng chảy của thông tin qua các cổng.

- Sử dụng các hàm kích hoạt khác như ReLU (Rectified Linear Unit), ít bị ảnh hưởng bởi tiêu biến gradient hơn sigmoid và tanh.

2. Bùng nổ Gradient (Exploding Gradient) Bùng nổ đạo hàm là hiện tượng ngược lại với tiêu biến gradient, khi gradient được lan truyền ngược qua các bước thời gian, nó có thể trở nên rất lớn do việc nhân liên tiếp các giá trị lớn (liên quan đến ma trận trọng số hoặc hàm kích hoạt). Điều này dẫn đến các gradient trở nên vô cùng lớn, gây ra lỗi số học hoặc làm mô hình dao động mà không hội tụ. Gradient lớn dẫn đến bước nhảy cập nhật quá lớn trong không gian trọng số, làm mất khả năng tối ưu.

Giải pháp cho vấn đề này là dùng Gradient Clipping. Thuật toán này đặt giới hạn trên cho gradient (ví dụ: cắt gradient về một giá trị tối đa). Nếu gradient vượt quá ngưỡng, nó sẽ bị "cắt"về giới hạn này để ngăn chặn giá trị quá lớn (Hình 4.5). Hơn nữa, sử dụng các biến thể RNN như LSTM hoặc GRU để giải quyết vấn đề này, vì chúng có cơ chế điều chỉnh dòng chảy thông tin, làm giảm khả năng bùng nổ gradient.

$\hat{g} \leftarrow \frac{\partial E}{\partial W}$
if $\| \hat{g} \| \geq threshold$ **then**
  $\hat{g} \leftarrow \frac{threshold}{\| \hat{g} \|} \hat{g}$
end if

Hình 4.5: Thuật toán Gradient clipping dùng giải quyết vấn đề bùng nổ đạo hàm

4.1.4 Các biến thể của RNN

Bi-RNN

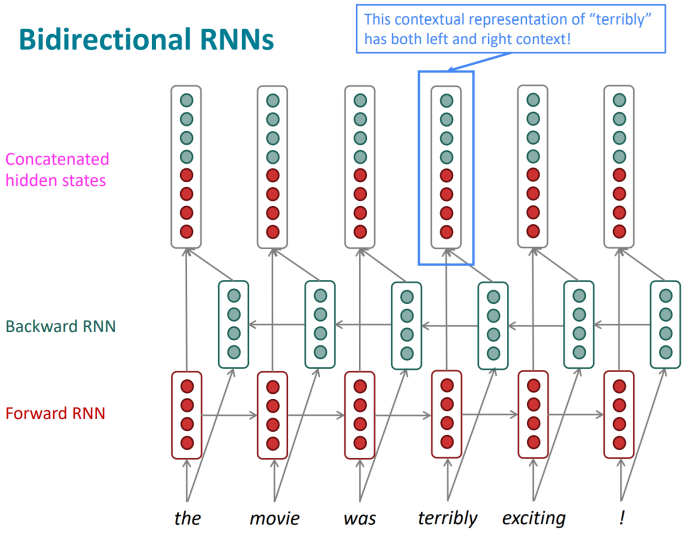
Mô hình RNN ở trên, chúng ta đã tập trung vào các RNN dựa trên các từ trong quá khứ để dự đoán từ tiếp theo trong chuỗi. Tuy nhiên, cũng có thể đưa ra dự đoán dựa trên các từ trong tương lai bằng cách để mô hình RNN đọc ngược lại tập dữ liệu. Các nhà khoa học đã giới thiệu một mạng nơ-ron hai chiều (Bi-RNN: Bi-directional RNN); tại mỗi bước thời gian t, mạng này duy trì hai lớp ẩn, một lớp cho quá trình truyền từ trái sang phải và một lớp cho quá trình truyền từ phải sang trái. Để duy trì hai lớp ẩn này tại bất kỳ thời điểm nào, mạng yêu cầu gấp đôi dung lượng bộ nhớ cho trọng số và các tham số bias.

Kết quả phân loại cuối cùng, $\hat{y}_t$, được tạo ra bằng cách kết hợp cả hai lớp ẩn của RNN. Hình 4.6 minh họa kiến trúc mạng hai chiều và các Công thức 4.5 và 4.6 mô tả công thức toán học việc thiết lập lớp ẩn RNN hai chiều. Sự khác biệt duy nhất giữa hai mối quan hệ này nằm ở hướng lan truyền qua tập dữ liệu. Công thức 4.7 chỉ ra mối quan hệ phân loại được sử dụng để dự đoán từ tiếp theo bằng cách tổng hợp các biểu diễn từ trong quá khứ và tương lai.

$$\vec{h}_t = f\left(\vec{W}x_t + \vec{V}\vec{h}_{t-1} + \vec{b}\right) \tag{4.5}$$

$$\overleftarrow{h}_t = f\left(\overleftarrow{W}x_t + \overleftarrow{V}\overleftarrow{h_{t+1}} + \overleftarrow{b}\right) \tag{4.6}$$

$$\hat{y}_t = g\left(Uh_t + c\right) = g\left(U\left[\vec{h}_t; \overleftarrow{h}_t\right] + c\right) \tag{4.7}$$



Hình 4.6: Mô hình Bi-RNN

Lưu ý khi dùng Bi-RNN:

- Bi-RNN chỉ được áp dụng khi biết toàn bộ chuỗi đầu vào.

- Không áp dụng được cho mô hình ngôn ngữ (Language Modeling - LM), vì trong LM chỉ có ngữ cảnh ở phía trái.

- Nếu chúng ta có toàn bộ chuỗi đầu vào thì nên áp dụng ngữ cảnh hai chiều. Ví dụ như mô hình BERT (Bidirectional Encoder Representations from Transformers) là một hệ thống tiền huấn luyện mạnh mẽ, biểu diễn ngữ cảnh được xây dựng dựa trên tính hai chiều.

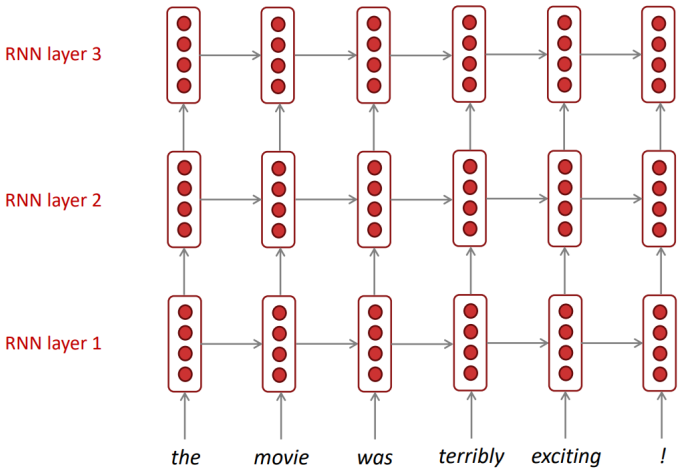
Mô hình RNN nhiều tầng - deep RNN

RNN cũng có thể được mở rộng thành nhiều tầng (multi-layered). Hình 4.7 minh họa một RNN nhiều tầng, trong đó mỗi lớp bên dưới cung cấp đầu ra cho lớp kế tiếp. Như được trình bày trong hình, trong kiến trúc mạng này, tại bước thời gian t, mỗi nơ-ron trung gian nhận một tập tham số từ bước thời gian trước đó (trong cùng một lớp RNN).

Để xây dựng một RNN sâu (Deep RNN) với L lớp, các mối quan hệ trên được điều chỉnh thành các mối quan hệ trong Công thức 4.8, trong đó đầu vào cho mỗi nơ-ron trung gian tại mức i là đầu ra của RNN tại lớp $i-1$ ở cùng bước thời gian t. Đầu ra $\hat{y}_t$ tại mỗi bước thời gian là kết quả của việc lan truyền các tham số đầu vào qua tất cả các lớp ẩn (Công thức 4.9).

$$\vec{h}_t^{(i)} = f\left(\vec{W}^{(i)}\vec{h}_t^{(i-1)} + \vec{V}^{(i)}\vec{h}_{t-1}^{(i)} + \vec{b}^{(i)}\right) \tag{4.8}$$

$$\hat{y}_t = g(Uh_t + c) = g\left(U\vec{h}_t^{(L)} + c\right) \tag{4.9}$$



Hình 4.7: Mô hình RNN nhiều tầng

Mô hình RNN nhiều tầng trong thực tế:

- RNN nhiều tầng hoặc RNN xếp chồng cho phép mô hình tính toán các biểu diễn phức tạp hơn. Mô hình này hoạt động tốt hơn so với mô hình chỉ sử dụng một lớp với các mã hóa có số chiều lớn. Các RNN ở tầng dưới nên tính toán các đặc trưng mức thấp hơn, trong khi các RNN ở lớp trên tính toán các đặc trưng mức cao hơn.

- Các mô hình RNN hiệu suất cao thường chứa nhiều tầng (nhưng không sâu bằng mạng tích chập hoặc mạng truyền thẳng). Ví dụ trong một nghiên cứu năm 2017, Britz và cộng sự phát hiện rằng với bài toán Dịch Máy khi dùng 2 đến 4 tầng là tốt nhất cho RNN mã hóa (encoder), và 4 tầng là tốt nhất cho RNN giải mã (decoder).

- Các mạng dựa trên mô hình Transformer (ví dụ: BERT) thường sâu hơn, dùng 12 hoặc 24 lớp.

4.2 Mô hình GRU - Gated Recurrent Units

Ngoài các thảo luận ở trên, RNN đã được chứng minh là hoạt động hiệu quả hơn khi sử dụng các đơn vị phức tạp hơn để kích hoạt. Chúng ta đã thảo luận về các phương pháp chuyển từ trạng thái ẩn h_{t-1} sang h_t thông qua một phép biến đổi affine và một hàm phi tuyến tính theo từng phần tử. Trong phần này, chúng ta sẽ tìm hiểu về việc sử dụng các hàm kích hoạt có cổng (gated activation functions), từ đó sửa đổi kiến trúc của RNN.

Mặc dù về lý thuyết, RNN có thể nắm bắt được các mối quan hệ phụ thuộc dài hạn, nhưng trên thực tế việc huấn luyện để đạt được điều này là rất khó khăn. Các đơn vị có cổng được thiết kế để duy trì thông tin một cách lâu dài hơn, từ đó giúp RNN dễ dàng nắm bắt các mối quan hệ phụ thuộc dài hạn hơn.

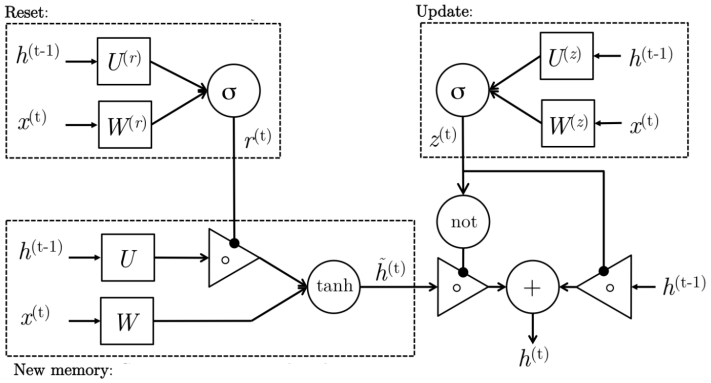
Về mặt toán học cách mà một GRU sử dụng h_{t-1} và x_t để tạo ra trạng thái ẩn tiếp theo h_t. Cụ thể, công thức toán học của mô hình này như sau:

$$z_t = \sigma\left(W^{(z)}x_t + U^{(z)}h_{t-1}\right) \tag{4.10}$$

$$r_t = \sigma\left(W^{(r)}x_t + U^{(r)}h_{t-1}\right) \tag{4.11}$$

$$\tilde{h}_t = \tanh\left(r_t \circ Uh_{t-1} + Wx_t\right) \tag{4.12}$$

$$h_t = (1 - z_t) \circ \tilde{h}_t + z_t \circ h_{t-1} \tag{4.13}$$



Hình 4.8: Chi tiết trong mô hình GRU

Các công thức trên có thể được xem như bốn bước hoạt động cơ bản của GRU, và chúng mang lại những diễn giải trực quan khiến mô hình này trở nên hấp dẫn (xem Hình 4.8):

- Cổng New memory: Một bộ nhớ mới $\tilde{h}_t$ là sự hợp nhất của một từ đầu vào mới x_t với trạng thái ẩn trước đó h_{t-1}. Theo cách hình dung, giai đoạn này giống như người biết cách kết hợp một từ mới được quan sát với trạng thái ẩn quá khứ h_{t-1} để tóm tắt từ mới này trong bối cảnh của quá khứ ngữ cảnh dưới dạng vector $\tilde{h}_t$.

- Cổng Reset: Tín hiệu đặt lại r_t có nhiệm vụ xác định mức độ quan trọng của h_{t-1} đối với việc tóm tắt $\tilde{h}_t$. Cổng đặt lại có khả năng hoàn toàn bỏ qua trạng thái ẩn trước đó nếu thấy rằng h_{t-1} không liên quan đến việc tính toán bộ nhớ mới.

- Cổng Update: Tín hiệu cập nhật z_t chịu trách nhiệm xác định mức độ của h_{t-1} cần được mang tới trạng thái tiếp theo. Ví dụ, nếu $z_t \approx 1$, thì h_{t-1} hầu như được sao chép hoàn toàn vào h_t. Ngược lại, nếu $z_t \approx 0$, thì phần lớn bộ nhớ mới $\tilde{h}_t$ sẽ được chuyển tiếp tới trạng thái ẩn tiếp theo.

- Trạng thái ẩn h_t cuối cùng được tạo ra bằng cách sử dụng đầu vào ẩn trước đó h_{t-1} và bộ nhớ mới $\tilde{h}_t$ với sự điều chỉnh từ cổng cập nhật.

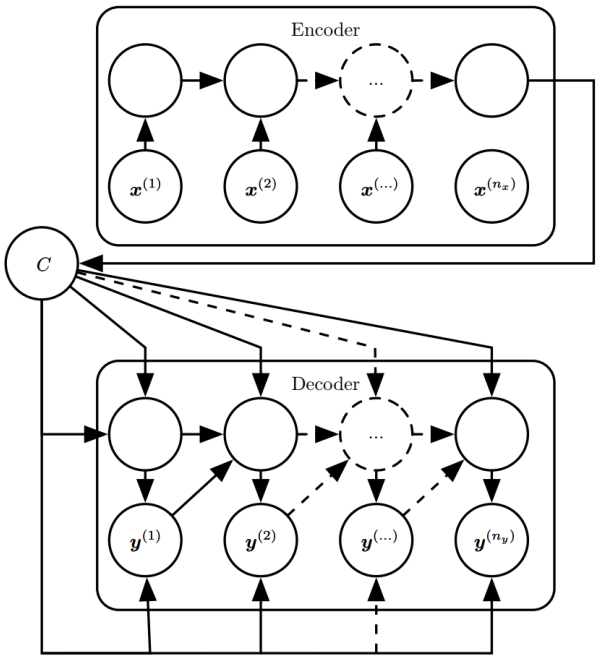
Điều quan trọng cần lưu ý là để huấn luyện một GRU, chúng ta cần học tất cả các tham số khác nhau: $W, U, W^{(r)}, U^{(r)}, W^{(z)}, U^{(z)}$. Những tham số này tuân theo quy trình lan truyền ngược (backpropagation) mà chúng ta đã trình bày phần trước.

Mô hình Sequece-to-sequence

Chúng ta đã thấy trong Hình 4.1 cách một RNN có thể ánh xạ một chuỗi đầu vào sang một vector có kích thước cố định. Trong Hình 4.2, chúng ta thấy cách một RNN ánh xạ một vector có kích thước cố định sang kí tự. Và trong mô hình RNN, một số bài toán như POS, NER sử dụng RNN ánh xạ một chuỗi đầu vào sang một chuỗi đầu ra có cùng độ dài.

Trong phần này, chúng tôi trình bày cách một RNN có thể được huấn luyện để ánh xạ một chuỗi đầu vào sang một chuỗi đầu ra mà không nhất thiết phải có cùng độ dài. Điều này thường xuất hiện trong nhiều ứng dụng, chẳng hạn như nhận diện giọng nói, dịch máy và trả lời câu hỏi, trong đó chuỗi đầu vào và đầu ra trong tập huấn luyện thường không có cùng độ dài (mặc dù chiều dài của chúng có thể liên quan đến nhau). Chúng ta thường gọi chuỗi đầu vào của RNN là "ngữ cảnh"(context). Mục tiêu là tạo ra một biểu diễn của ngữ cảnh này, C. Ngữ cảnh C có thể là một vector hoặc một chuỗi các vector tóm tắt chuỗi đầu vào $\mathbf{X} = \left(x^{(1)}, \ldots, x^{(n_x)}\right)$.

Kiến trúc RNN đơn giản nhất để ánh xạ một chuỗi có độ dài biến đổi sang một chuỗi khác cũng có độ dài biến đổi được đề xuất lần đầu bởi Cho và cộng sự [5], và ngay sau đó bởi Sutskever [6]. Hai nghiên cứu này độc lập phát triển kiến trúc này và là những nghiên cứu đầu tiên đạt được kết quả tiên tiến trong dịch máy sử dụng phương pháp này. Hệ thống đầu tiên dựa trên việc đánh giá các đề xuất được tạo bởi một hệ thống dịch máy khác, trong khi hệ thống thứ hai sử dụng một mạng hồi tiếp độc lập để tạo ra bản dịch. Các tác giả này lần lượt gọi kiến trúc này, được minh họa trong Hình 4.9, là kiến trúc mã hóa-giải mã (encoder-decoder) hay kiến trúc chuỗi sang chuỗi (sequence-to-sequence).



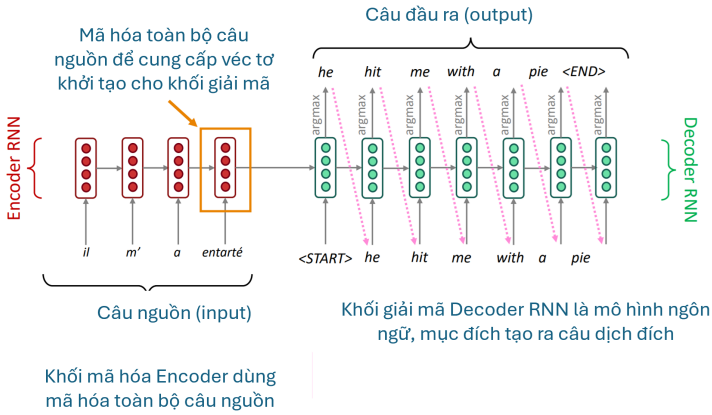
Hình 4.9: Mô hình Sequence to Sequence RNN

Ý tưởng của mô hình này rất đơn giản như sau:

- Bộ mã hóa (encoder): xử lý chuỗi đầu vào. Bộ mã hóa tạo ra ngữ cảnh C, thường là một hàm đơn giản của trạng thái ẩn cuối cùng.
- Bộ giải mã (decoder): sử dụng vector có độ dài cố định để tạo ra chuỗi đầu ra $\mathbf{Y} = \left(y^{(1)}, \ldots, y^{(n_y)}\right)$. Sự đổi mới của loại kiến trúc này so với các kiến trúc được trình bày trước đó trong phần này là độ dài n_x và n_y có thể khác nhau, trong khi các kiến trúc trước đây yêu cầu $n_x = n_y = \tau$. Trong kiến trúc sequence-to-sequence, hai RNN được huấn luyện cùng nhau để tối đa hóa trung bình của $\log P\left(y^{(1)}, \ldots, y^{(n_y)} \mid x^{(1)}, \ldots, x^{(n_x)}\right)$ trên tất cả các cặp x và y trong tập huấn luyện. Trạng thái cuối cùng h_{n_x} của RNN mã hóa thường được sử dụng làm đại diện C của chuỗi đầu vào, được cung cấp làm đầu vào cho RNN giải mã.

Nếu ngữ cảnh C là một vector, thì RNN giải mã đơn giản là một RNN từ vector sang chuỗi (như Hình 4.10). Như chúng ta đã thấy, có ít nhất hai cách để một RNN từ vector sang chuỗi nhận đầu vào. Đầu vào có thể được cung cấp dưới dạng trạng thái ban đầu của RNN, hoặc đầu vào có thể được kết nối

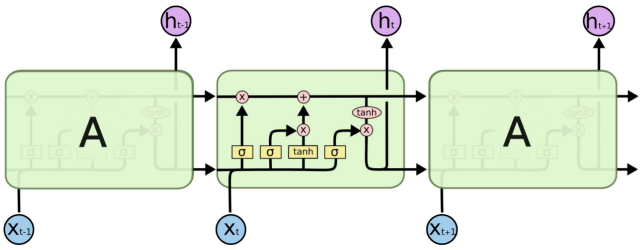
với các đơn vị ẩn tại mỗi bước thời gian. Hai cách này cũng có thể được kết hợp. Không có ràng buộc nào yêu cầu bộ mã hóa phải có kích thước lớp ẩn giống với bộ giải mã.



Hình 4.10: Mô hình Sequence to Sequence với bài toán dịch máy từ tiếng Pháp sang tiếng Anh

Một hạn chế của kiến trúc này là khi ngữ cảnh C được RNN mã hóa có kích thước quá nhỏ để tóm tắt chính xác một chuỗi dài. Hiện tượng này đã được Bahdanau [7] quan sát trong bối cảnh dịch máy. Họ đề xuất làm cho C trở thành một chuỗi có độ dài biến đổi thay vì một vector có kích thước cố định. Ngoài ra, các nhà nghiên cứu đã giới thiệu cơ chế attention giúp học cách liên kết các phần tử của chuỗi C với các phần tử của chuỗi đầu ra.

4.3 Mô hình LSTM - Long Short Term Memory



Hình 4.11: Mô hình LSTM

LSTM (Long-Short-Term-Memories) là mô hình cải tiến khác của RNN, khác một chút so với GRU. Ý nghĩa của LSTM tương tự như đối với GRU, tuy nhiên kiến trúc của các đơn vị này có sự khác biệt. Hình 4.11 mô tả sơ đồ tổng quát của LSTM. Chúng ta xem xét công thức toán học của các đơn vị LSTM trước khi đi sâu vào trực giác đằng sau thiết kế này:

$$i_t = \sigma\left(W^{(i)}x_t + U^{(i)}h_{t-1}\right) \quad \text{(Cổng đầu vào - Input gate)} \tag{4.14}$$

$$f_t = \sigma\left(W^{(f)}x_t + U^{(f)}h_{t-1}\right) \quad \text{(Cổng quên - Forget gate)} \tag{4.15}$$

$$o_t = \sigma\left(W^{(o)}x_t + U^{(o)}h_{t-1}\right) \quad \text{(Cổng đầu ra/tiếp xúc - Output/Exposure gate)} \tag{4.16}$$

$$\tilde{c}_t = \tanh\left(W^{(c)}x_t + U^{(c)}h_{t-1}\right) \quad \text{(Bộ nhớ mới - New memory cell)} \tag{4.17}$$

$$c_t = f_t \circ c_{t-1} + i_t \circ \tilde{c}_t \quad \text{(Bộ nhớ cuối cùng - Final memory cell)} \tag{4.18}$$

$$h_t = o_t \circ \tanh(c_t) \quad \text{(Trạng thái ẩn - Hidden state)} \tag{4.19}$$

Chúng ta có thể hiểu về cấu trúc của một LSTM bằng cách xem xét kiến trúc của nó (xem Hình 4.12) qua các bước sau:

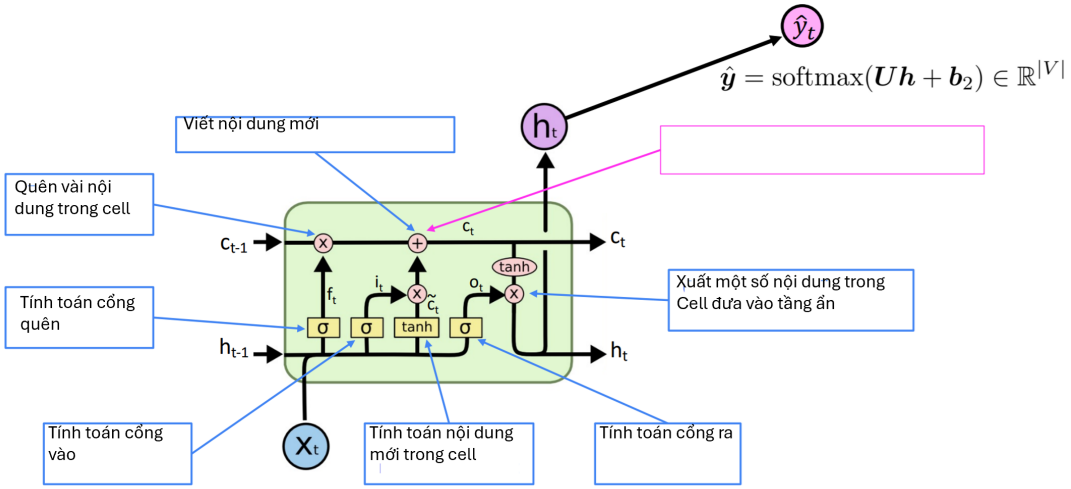
- Tạo bộ nhớ mới: Giai đoạn này tương tự như giai đoạn tạo bộ nhớ mới trong GRU. Chúng ta sử dụng từ đầu vào x_t và trạng thái ẩn trước đó h_{t-1} để tạo ra một bộ nhớ mới $\tilde{c}_t$, trong đó bao gồm các đặc điểm của từ mới x_t.

- Cổng đầu vào (Input Gate): Chúng ta thấy rằng giai đoạn tạo bộ nhớ mới không kiểm tra xem từ mới có thực sự quan trọng trước khi tạo bộ nhớ mới – đây chính là chức năng của cổng đầu vào. Cổng đầu vào sử dụng từ đầu vào và trạng thái ẩn trước đó để xác định liệu đầu vào có đáng để lưu giữ hay không và từ đó quyết định cách cổng bộ nhớ mới. Vì vậy, nó tạo ra i_t như một chỉ số của thông tin này.

- Cổng quên (Forget Gate): Cổng này tương tự như cổng đầu vào, ngoại trừ việc nó không xác định mức độ quan trọng của từ đầu vào – thay vào đó, nó đánh giá xem bộ nhớ trước đó có hữu ích cho việc tính toán bộ nhớ hiện tại hay không. Do đó, cổng quên xem xét từ đầu vào và trạng thái ẩn trước đó và tạo ra f_t.

- Tạo bộ nhớ cuối cùng: Giai đoạn này đầu tiên sử dụng thông tin từ cổng quên f_t và quên đi bộ nhớ trước đó c_{t-1}. Tương tự, nó cũng sử dụng thông tin từ cổng đầu vào i_t và cổng bộ nhớ mới $\tilde{c}_t$. Sau đó, nó cộng hai kết quả này để tạo ra bộ nhớ cuối cùng c_t.

- Cổng đầu ra/tiếp xúc (Output Gate): Đây là một cổng không tồn tại rõ ràng trong GRU. Mục đích của nó là để tách bộ nhớ cuối cùng khỏi trạng thái ẩn. Bộ nhớ cuối cùng c_t chứa rất nhiều thông tin, không nhất thiết phải được lưu trong trạng thái ẩn. Các trạng thái ẩn được sử dụng trong mọi cổng của LSTM, vì vậy cổng này đánh giá phần nào của bộ nhớ c_t cần được đưa ra/lưu giữ trong trạng thái ẩn h_t. Tín hiệu mà nó tạo ra được chỉ định là o_t, và tín hiệu này được sử dụng để cổng hàm tanh từng phần của bộ nhớ.



Hình 4.12: Trực quan một Cell trong mô hình LSTM

Thành công của LSTM trong thực tế:

- Trong giai đoạn 2013–2015: LSTM bắt đầu đạt được kết quả tốt nhất trong lĩnh vực bao gồm nhận diện chữ viết tay, nhận diện giọng nói, dịch máy, phân tích cú pháp, tạo chú thích hình ảnh, cũng như các mô hình ngôn ngữ. LSTM trở thành phương pháp chủ đạo cho hầu hết các bài toán xử lý ngôn ngữ tự nhiên (NLP).

- Giai đoạn hiện tại 2019–2024: các mô hình Transformer đã trở thành tiêu chuẩn cho mọi bài toán. Ví dụ trong cuộc thi về dịch máy: Năm 2014, không có hệ thống dịch máy sử dụng mạng nơ-ron nào. Đến năm 2016 trong báo cáo tổng kết thống kê mô hình RNN được sử dụng 44 lần (kể cả các hệ thống giành chiến thắng). Trong khi đó năm 2019, RNN chỉ được 7 đội sử dụng, Transformer được dùng 105 lần.

4.4 Cơ chế chú ý

Mô hình Sequence-to-Sequence (Seq2Seq) trong dịch máy, dựa trên kiến trúc Encoder-Decoder với RNN/LSTM, gặp phải một số hạn chế quan trọng khi xử lý các câu dài hoặc phức tạp. Một trong những nhược điểm chính là nghẽn cổ chai thông tin, do toàn bộ nội dung câu nguồn bị nén vào một vector ngữ cảnh (context vector) cố định ở lớp cuối của Encoder, khiến thông tin bị mất mát khi câu quá dài. Bên cạnh đó, mô hình này không bảo toàn tốt cấu trúc và vị trí từ trong câu, dẫn đến khó khăn trong việc nắm bắt các phụ thuộc xa (long-range dependencies). Khi độ dài câu tăng, hiệu suất dịch suy giảm do trạng thái ẩn của RNN/LSTM bị suy yếu hoặc bão hòa qua nhiều bước.

Để khắc phục, cơ chế Attention được đề xuất, cho phép Decoder tập trung động vào các phần liên quan của câu nguồn tại mỗi bước dịch. Thay vì phụ thuộc vào một vector ngữ cảnh duy nhất, Attention tính toán trọng số (attention weights) để kết hợp tất cả các trạng thái ẩn của Encoder, ưu tiên thông tin quan trọng. Cách tiếp cận này giúp giải quyết vấn đề nghẽn cổ chai, bảo toàn ngữ cảnh theo vị trí, và cải thiện khả năng xử lý các phụ thuộc xa. Nhờ đó, chất lượng dịch được nâng cao ngay cả với câu dài, đồng thời tạo tiền đề cho các mô hình tiên tiến hơn như Transformer dựa trên Self-Attention

Cơ chế chú ý (attention) trong học máy giúp cho mô hình có thể tập trung nhiều hơn vào các từ khóa chính hay cụm từ mang nhiều thông tin thông qua việc học các trọng số chú ý trong câu. Sau đó việc tổng hợp thông tin của cả câu được tính qua trung bình cộng có trọng số của tất cả các từ trong câu. Thời gian gần đây có nhiều nghiên cứu sử dụng cơ chế chú ý trong học sâu được phát triển dựa vào ý tưởng này. Các mô hình này được ứng dụng đầu tiên trong dịch máy.

Một trong những ý tưởng của cơ chế chú ý đó là học để căn chỉnh (learn to align) [8]. Xuất phát từ ý tưởng tổng hợp thông tin từ câu nguồn và các từ được dịch trước đó để tính ra những phần quan trọng để dự đoán từ tiếp theo. Thay vì chỉ sử dụng ngữ cảnh của lớp cuối cùng của lớp mã hóa (Encoder), tác giả sử dụng tất cả các đầu ra của từng tế bào qua từng bước (timestep) của câu nguồn kết hợp với trọng số chú ý của câu nguồn ở mỗi bước giải mã để tổng hợp ra véc tơ ngữ cảnh $\mathbf{c}_i$. Véc tơ này được dùng để dự đoán ra từ tiếp theo trong phần giải mã (decoder). Hình 4.13 mô tả cơ chế chú ý của Bahdanau. Công thức cụ thể như sau:

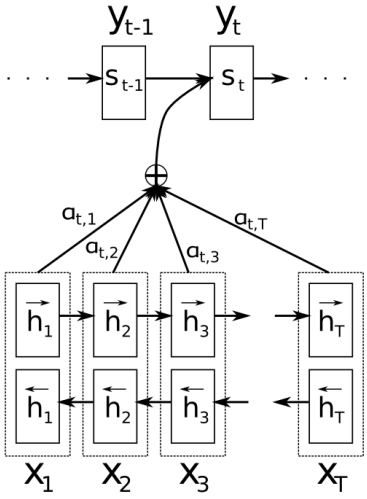
$$P(y_i|y_1, y_2, ..., y_{i-1}, \mathbf{x}) = g(y_{i-1}, \mathbf{s}_i, \mathbf{c}_i) \tag{4.20}$$

$$\mathbf{s}_i = f(\mathbf{s}_{i-1}, y_{i-1}, \mathbf{c}_i), \tag{4.21}$$

trong đó $\mathbf{x}$ là chuỗi các từ trong văn bản gốc (câu nguồn) có độ dài T, $\mathbf{y}$ là chuỗi các từ trong văn bản dịch (câu đích) có độ dài m

$\mathbf{x} = (x_1, x_2, ..., x_T)$

$\mathbf{y} = (y_1, y_2, ..., y_m)$



Hình 4.13: Mô hình attention trong bài toán dịch máy của Bahdanau

Trước tiên véc tơ ngữ cảnh $\mathbf{c}_i$ được tính qua tổng trọng số của các $\mathbf{h}_j$

$$\mathbf{c}_i = \sum_{j=1}^{T_x} \alpha_{ij} \mathbf{h}_j \tag{4.22}$$

trong đó, $\mathbf{h}_j = [\overrightarrow{\mathbf{h}_j}^T; \overleftarrow{\mathbf{h}_j}^T]^T$, trọng số α_{ij} của $\mathbf{h}_j$ được tính như sau:

$$\alpha_{ij} = \frac{exp(e_{ij})}{\sum_{k=1}^{T_x} exp(e_{ik})} \tag{4.23}$$

$$e_{ij} = a(\mathbf{s}_{i-1}, \mathbf{h}_j), \tag{4.24}$$

trong đó a là công thức gióng (alignment) (dạng đơn giản của a có thể là multilayer perception) cho biết độ tương quan của từ thứ j của lớp mã hóa so với đầu ra ở vị trí thứ i của lớp giải mã.

$$a(\mathbf{s}_{i-1}, \mathbf{h}_j) = \mathbf{v}_a^T tanh(\mathbf{W}_a \mathbf{s}_{i-1} + \mathbf{U}_a \mathbf{h}_j), \tag{4.25}$$

trong đó: $\mathbf{W}_a \in \mathbb{R}^{n.n}$, $\mathbf{U}_a \in \mathbb{R}^{n.2n}$, $\mathbf{v}_a \in \mathbb{R}^n$ là các ma trận trọng số.

Thực chất cơ chế chú ý giúp cho mô hình tập trung vào các phần quan trọng trên dữ liệu qua mô hình gióng (alignment model a) để tính các trọng số gióng (alignment score e_{ij}), sau đó chuẩn hóa để cho ra trọng số chú ý (attention score α_{ij}). Trong bài toán dịch máy, mỗi lần dự đoán ra từ tiếp theo y_i các trọng số α_{ij} được tính lại mức độ quan trọng trong câu đầu vào x.

Có nhiều cách để tính $a(\mathbf{s}_{i-1}, \mathbf{h}_j)$. Theo Graves [**?**] công thức tính như sau:

$$a(\mathbf{s}_{i-1}, \mathbf{h}_j) = cosin(\mathbf{s}_{i-1}, \mathbf{h}_j). \tag{4.26}$$

theo Luong [9]:

$$a(\mathbf{s}_{i-1}, \mathbf{h}_j) = \mathbf{s}_{i-1}^T \mathbf{W}_a \mathbf{h}_j \tag{4.27}$$

và

$$a(\mathbf{s}_{i-1}, \mathbf{h}_j) = \mathbf{s}_{i-1}^T \mathbf{h}_j \tag{4.28}$$

Ý tưởng tiếp theo là chú ý địa phương và chú ý toàn cục [9]. Chú ý toàn cục lấy ý tưởng học để gióng (learn to align), còn chú ý cục bộ giúp cho mô hình học ra vị trí cần chú ý. Gần đây nhất là ý tưởng về tự chú ý (self-attention) [2] với mục đích học ra mối tương quan của từ đang xét với các từ đứng trước nó. Cơ chế tự chú ý được mô tả chi tiết trong mô hình Transformer bên dưới.

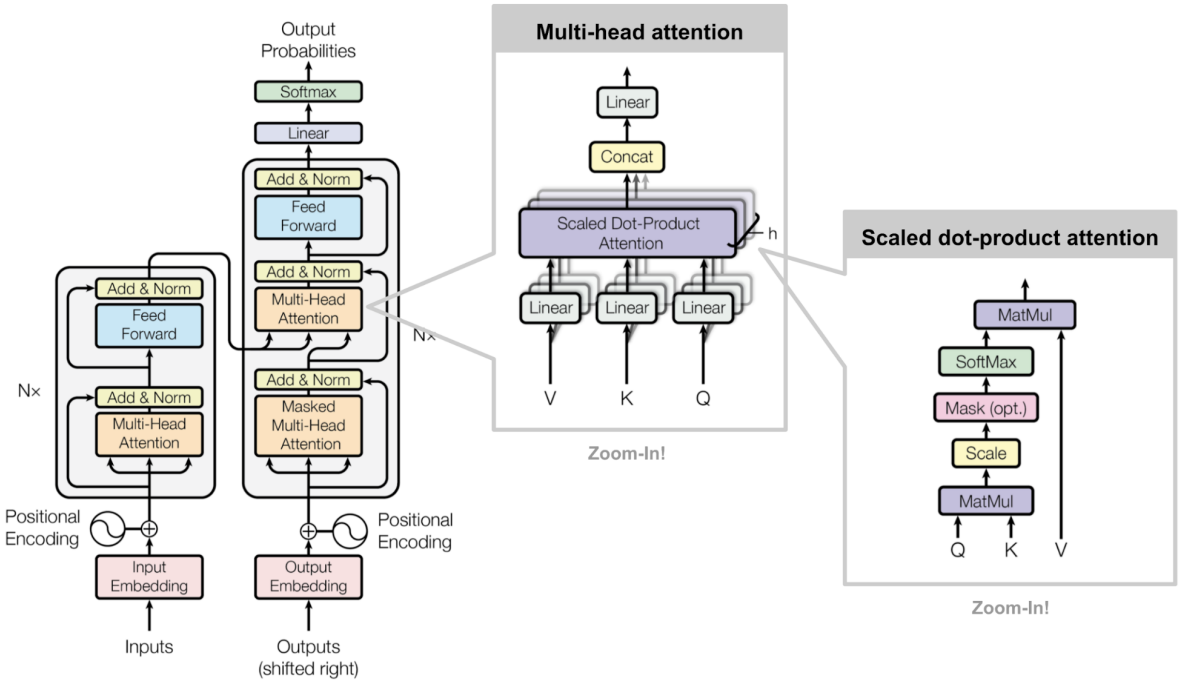
4.5 Transformer

4.5.1 Giới thiệu về mô hình Transformer

Phần này chúng tôi sẽ trình bày về mô hình Transformer, một trong những kiến trúc quan trọng và mang tính cách mạng trong lĩnh vực trí tuệ nhân tạo, đặc biệt là xử lý ngôn ngữ tự nhiên (NLP). Transformer không chỉ là nền tảng của hầu hết các ứng dụng NLP hiện đại mà còn được ứng dụng rộng rãi trong nhiều lĩnh vực khác như thị giác máy tính, xử lý âm thanh, và thậm chí là sinh học tính toán. Kiến trúc của Transformer đã được giới thiệu lần đầu tiên vào năm 2017 trong bài báo "Attention is All You Need", với cơ chế self-attention độc đáo giúp mô hình có khả năng nắm bắt ngữ cảnh tốt hơn so với các phương pháp truyền thống như RNN hay CNN.

Một khía cạnh đáng chú ý khác của Transformer là tốc độ xử lý nhanh chóng nhờ việc chỉ sử dụng các phép nhân ma trận đơn giản. Điều này tạo nên sự khác biệt lớn so với RNN, vốn thường bị chậm lại do phải đợi kết quả từ trạng thái trước đó trước khi có thể tính toán trạng thái tiếp theo. Transformer đã loại bỏ hoàn toàn hạn chế này, mang lại hiệu suất tính toán vượt trội. Nguyên nhân chính khiến Transformer trở nên phổ biến không chỉ nằm ở khả năng mô hình hóa hiệu quả hơn, mà quan trọng hơn là nhờ tốc độ xử lý cực kỳ nhanh chóng của nó.

Transformer [2] là mô hình đang được sử dụng phổ biến trong các bài toán NLP trong mấy năm gần đây do sự mạnh mẽ của nó trong việc song song hóa tính toán và khả năng nắm bắt được sự phụ thuộc xa nhờ cơ chế Self-attention. Mô hình transformer đưa ra phục vụ cho bài toán dịch máy bao gồm hai phần mã hóa và giải mã cũng gần giống RNN tuy nhiên có điểm khác biệt là các từ đầu vào được đưa vào mô hình cùng một lúc (hình 4.14). Vì thế không còn khái niệm bước (timestep) trong Transformer mà thay vào đó là cơ chế self-attention.



Hình 4.14: Kiến trúc Transformer[2]

Phần mã hóa

Đầu tiên các từ được biểu diễn bằng một véc tơ từ nhúng word-embedding(WE). Nhưng vì các từ được đưa vào đồng thời mà lại muốn biểu diễn được ngữ cảnh của từ (do các từ ở vị trí khác nhau sẽ có nghĩa khác nhau) cho nên Transformer có thêm phần mã hóa vị trí Positional-Encoding(PE) để thêm thông tin về vị trí của từ. Sau đó cộng hai véc tơ PE và WE để cho ra biểu diễn của một từ. Công thức tính PE của từ thứ i trong câu như sau:

$$\begin{aligned} PE_{(pos,2i)} &= \sin \frac{pos}{10000^{2i/d_{model}}} \\ PE_{(pos,2i+1)} &= \cos \frac{pos}{10000^{2i/d_{model}}} \end{aligned} \tag{4.29}$$

trong đó pos là vị trí của từ trong câu, PE là giá trị phần tử thứ i trong *embedding* có độ dài d_{model}

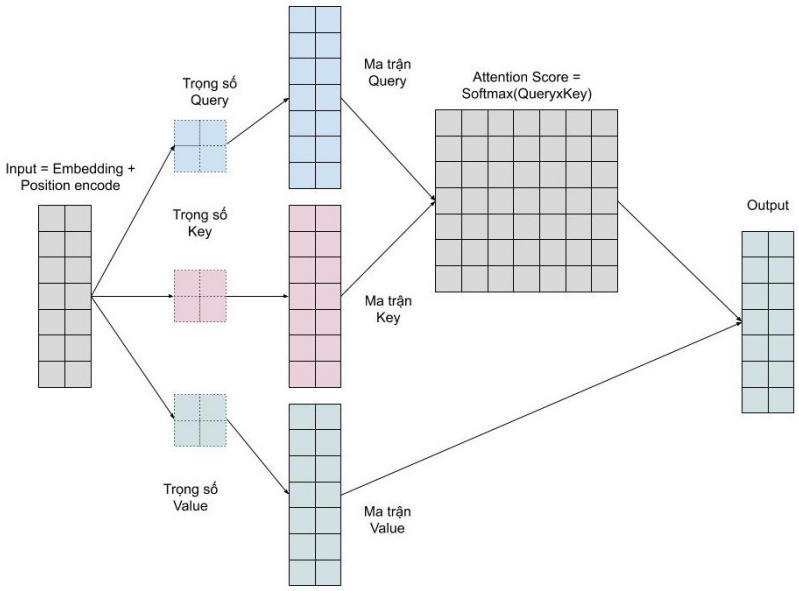
Multihead-attention là lớp mới tạo ra sự khác biệt của mô hình transformer với các mô hình khác như RNN và LSTM. Thực chất của Multihead-attention là self-attention nhưng để mô hình chú ý đến nhiều phần khác nhau thì tác giả sử dụng nhiều lớp self-attention. Self-attention là cơ chế giúp cho mô hình có thể thấy được một từ có mức độ chú ý như thế nào tới các từ còn lại. Đầu tiên, với mỗi từ cần tạo ra 3 véc tơ: *query*, *key*, *value*. Ba véc tơ này được tạo ra bằng cách nhân ma trận biểu diễn các từ đầu vào với ma trận học tương ứng. *Query* là véc tơ dùng để chứa thông tin của từ được tìm kiếm, so sánh. Giống như là câu *query* của Google search. *Key* là véc tơ dùng để biểu diễn thông tin các từ trong câu so sánh với từ cần tìm kiếm ở trên. Ví dụ, như nội dung của các website sẽ so sánh với từ khóa mà bạn tìm kiếm. *Value* là véc tơ biểu diễn nội dung, ý nghĩa của các từ. Giá trị như nội dung trang web được hiển thị cho người dùng sau khi tìm kiếm (Hình 4.15[1]). Self-attention được lặp nhiều lần gọi là multihead-attention. Công thức tính như sau:

$$Attention(Q, K, V) = softmax(\frac{QK^T}{\sqrt{d_k}})V \tag{4.30}$$

d_k là số chiều của Key.

Thông qua Multihead-attention, nó cho phép mô hình tập trung vào các ngữ cảnh khác nhau như: tầm quan trọng của một từ với các từ trước của nó, hay với các từ sau của nó, hay là các từ liên quan tới nó. Trong Transformer, các lớp kết nối dư thừa và chuẩn hóa (residual connection và normalization)

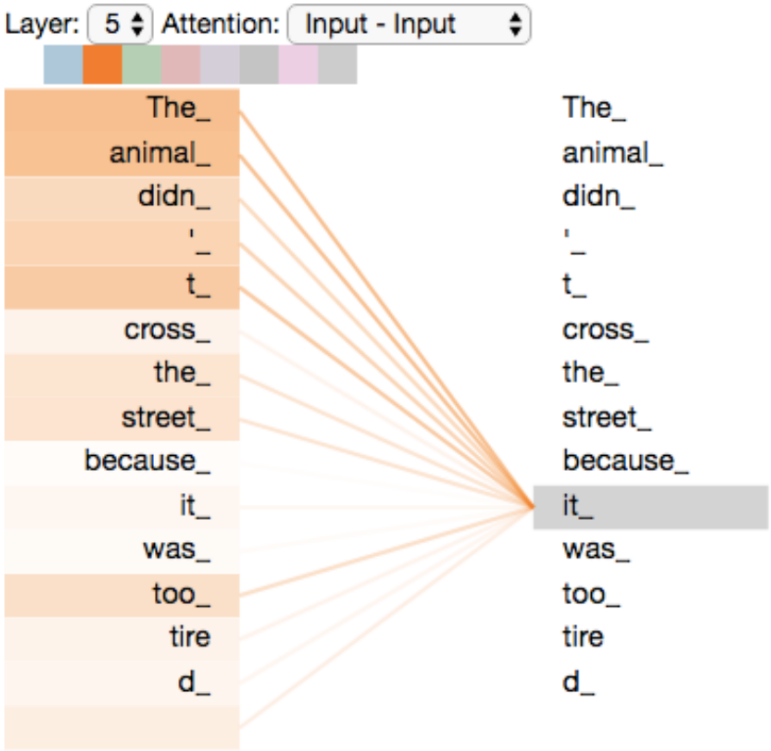
[1]https://pbcquoc.github.io/transformer/



Hình 4.15: Quá trình tính sự chú ý

được sử dụng ở mỗi lớp con để giúp mô hình nhanh hội tụ và tránh mất mát thông tin trong quá trình huấn luyện.

Hình 4.16 ví dụ minh họa cơ chế Self-attention[2]. Đây là phương pháp mà Transformer sử dụng để đưa "sự hiểu biết" về các từ liên quan khác vào từ mà chúng ta đang xử lý. Ví dụ này là mã hóa từ "it"trong bộ mã hóa ở layer 5 (bộ mã hóa trên cùng trong ngăn xếp), một phần của cơ chế chú ý tập trung nhiều nhất vào "The Animal"(màu càng đậm thì mô tả trọng số chú ý càng lớn) và đưa một phần biểu diễn của nó vào trong quá trình mã hóa của "it".



Hình 4.16: Ví dụ về multihead-attention

Phần giải mã

[2]https://jalammar.github.io/illustrated-transformer/

Phần giải mã làm chức năng giải mã véc tơ đặc trưng của câu nguồn để đưa ra câu đích. Ở mô hình này phần giải mã nhận đầu vào là hai véc tơ *key* và *value* từ phần mã hóa. Kiến trúc của phần giải mã gần giống với phần mã hóa chỉ khác ở chỗ có thêm lớp Multihead-attention ở giữa để học mối quan hệ giữa từ đang được dịch và từ của câu nguồn.

Lớp *Masked* của Multihead-attention trong phần giải mã bản chất là multihead-attention, nó có chức năng dùng để mã hóa các từ của câu đích trong quá trình dịch. Để làm việc này thì đơn giản là chúng ta chỉ cần nhân với một véc tơ chứa các giá trị $0, 1$.

Trong giải mã có một Multihead-attention như nói ở trên có chức năng chú ý các từ ở mô hình mã hóa, lớp này nhận véc tơ *key* và *value* từ mô hình mã hóa, và đầu ra từ lớp của *masked* Multihead-attention. Lớp này với mục đích so sánh sự tương quan giữa từ đang được dịch với các từ nguồn. Hàm mất mát là hàm $cross - entropy$.

4.5.2 Demo về Transformer

Trong phần này chúng tôi trình bày đoạn code nhỏ đi dự đoán từ tiếp theo trong câu sử dụng mô hình nn.Transformer. Dữ liệu dùng để huấn luyện mô hình Transformer lấy từ Hugging Face Datasets.Đây là tập dữ liệu gồm 25.000 bài đánh giá phim từ IMDB, được gắn nhãn theo cảm tính (tích cực/tiêu cực). Các bài đánh giá đã được xử lý trước và mỗi bài đánh giá được mã hóa thành danh sách các chỉ mục từ (số nguyên)

Bước 1: Thiết lập các thư viện cần thiết

```
import torch
import torch.nn as nn
import torch.optim as optim
import numpy as np
import math
```

Bước 2: Định nghĩa Tokenizer và Vocabulary

```
# vi du vocabulary and tokenizer
vocab = ['i', 'am', 'a', 'student', 'teacher', 'you', 'are', 'learning', 'to',
'build', 'language', 'model']
vocab_size = len(vocab)
word_to_id = {word: idx for idx, word in enumerate(vocab)}
id_to_word = {idx: word for idx, word in enumerate(vocab)}

# Dinh nghia ham: tokenizer function
def tokenize(sentence):
    return [word_to_id[word] for word in sentence.lower().split() if word in word_to_id]

# Dinh nghia ham detokenize
def detokenize(tokens):
    return ' '.join([id_to_word[token] for token in tokens])
```

Bước 3: Xác định Positional Encoding

```
class PositionalEncoding(nn.Module):
    def __init__(self, d_model, max_len=5000):
        super(PositionalEncoding, self).__init__()
        pe = torch.zeros(max_len, d_model)
        position = torch.arange(0, max_len, dtype=torch.float).unsqueeze(1)
        div_term = torch.exp(torch.arange(0, d_model, 2).float() * (-math.log(10000.0) /
        d_model))
        pe[:, 0::2] = torch.sin(position * div_term)
        pe[:, 1::2] = torch.cos(position * div_term)
        self.pe = pe.unsqueeze(0).transpose(0, 1)

    def forward(self, x):
        return x + self.pe[:x.size(0), :]
```

Bước 4: Xây dựng mô hình Transformer

```
class NanoLLM(nn.Module):
    def __init__(self, vocab_size, embedding_dim, num_heads, num_layers):
        super(NanoLLM, self).__init__()
        self.embedding = nn.Embedding(vocab_size, embedding_dim)
        self.positional_encoding = PositionalEncoding(embedding_dim)
        self.transformer_blocks = nn.Transformer(d_model=embedding_dim,
        nhead=num_heads, num_encoder_layers=num_layers, num_decoder_layers=num_layers)
        self.fc_out = nn.Linear(embedding_dim, vocab_size)

    def forward(self, src, tgt):
```

```
        src = self.embedding(src) * math.sqrt(src.size(1))
        tgt = self.embedding(tgt) * math.sqrt(tgt.size(1))
        src = self.positional_encoding(src)
        tgt = self.positional_encoding(tgt)
        transformer_output = self.transformer_blocks(src, tgt)
        output = self.fc_out(transformer_output)
        return output
```

Bước 5: Sinh dữ liệu thử nghiệm

```
def generate_example_data():
    sentences = [
        'i am a student',
        'you are learning to build',
        'i am a teacher',
        'you are a student'
    ]
    data = []
    for sentence in sentences:
        tokens = tokenize(sentence)
        input_seq = tokens[:-1]
        target_seq = tokens[1:]
        data.append((torch.tensor(input_seq, dtype=torch.long),
        torch.tensor(target_seq, dtype=torch.long)))
    return data

training_data = generate_example_data()
```

Bước 6: Huấn luyện mô hình

```
# khoi tao sieu tham so Hyperparameters
embedding_dim = 32
num_heads = 2
num_layers = 2
learning_rate = 0.001
num_epochs = 100

# khoi tao model, loss function, and optimizer
model = NanoLLM(vocab_size, embedding_dim, num_heads, num_layers)
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=learning_rate)

# huan luyen mo hinh
for epoch in range(num_epochs):
    total_loss = 0
    for src, tgt in training_data:
        src = src.unsqueeze(1)
        tgt = tgt.unsqueeze(1)
        optimizer.zero_grad()
        output = model(src, tgt)
        loss = criterion(output.view(-1, vocab_size), tgt.view(-1))
        loss.backward()
        optimizer.step()
        total_loss += loss.item()

    if (epoch + 1) % 10 == 0:
        print(f'Epoch [{epoch + 1}/{num_epochs}],
        Loss:  {total_loss/len(training_data):.4f}')
```

Bước 7: Dự đoán

```
def predict_next_word(model, input_seq):
    model.eval()
    with torch.no_grad():
        tokens = tokenize(input_seq)
        input_tensor = torch.tensor(tokens, dtype=torch.long).unsqueeze(1)
        output = model(input_tensor, input_tensor)
        predicted_id = torch.argmax(output[-1, 0, :]).item()
        return id_to_word[predicted_id]

# Vi du sinh tu
input_sentence = "you"
predicted_word = predict_next_word(model, input_sentence)
print(f"Input: '{input_sentence}', Predicted next word: '{predicted_word}'")
```

4.6 Bài tập

Dưới đây là một số câu hỏi ôn tập kiến thức về mô hình RNN và LSTM dành cho sinh viên học môn Xử lý ngôn ngữ tự nhiên:

Bài 1: Hãy trình bày về mô hình RNN:

- RNN là viết tắt của từ gì? Tại sao RNN phù hợp cho dữ liệu tuần tự?
- Mô hình RNN hoạt động như thế nào để xử lý các chuỗi dữ liệu đầu vào?
- Giải thích vai trò của hàm kích hoạt (activation function) trong RNN.
- RNN có thể được sử dụng trong các bài toán nào?
- Trình bày nhược điểm chính của RNN khi xử lý các chuỗi dài?

Bài 2: Trình bày về mô hình LSTM

- LSTM là viết tắt của từ gì? Tại sao nó được xem là cải tiến so với RNN thông thường?
- Giải thích cấu trúc của một tế bào LSTM (LSTM cell), bao gồm: Input gate, Forget gate, Output gate, Cell state.
- LSTM giải quyết vấn đề gradient biến mất như thế nào?
- So sánh LSTM và GRU (Gated Recurrent Unit). GRU khác gì so với LSTM?

Bài 3

- Trong bài toán dịch máy (machine translation), RNN và LSTM được sử dụng như thế nào?
- Trình bày một số phương pháp sử dụng LSTM trong bài toán phân lớp văn bản?
- Hãy trình bày cách áp dụng LSTM vào bài toán nhận dạng thực thể có tên (NER).
- Nêu các trường hợp mà LSTM không hiệu quả bằng các mô hình hiện đại khác như Transformer.

Bài 4

- Giải thích sự khác biệt giữa Bidirectional LSTM và LSTM thông thường.
- Trong một bài toán trên text, làm thế nào để chọn số lượng lớp ẩn (hidden layers) và số lượng đơn vị ẩn (hidden units) trong LSTM?

Bài 5

Trong một bài toán chuỗi thời gian, làm thế nào để chuẩn hóa dữ liệu đầu vào cho RNN hoặc LSTM? Hãy thiết kế một kiến trúc LSTM để giải quyết bài toán phân loại văn bản.

Bài 6

Giải thích 3 cổng (gate) trong LSTM và ý nghĩa của chúng. Triển khai LSTM 1 lớp với PyTorch cho bài toán dự đoán chuỗi thời gian (input_size=10, hidden_size=32).

Bài 7

Mô tả cách chuẩn bị dữ liệu dạng chuỗi (sequence) cho LSTM trong PyTorch. Ví dụ với chuỗi độ dài 100, chia thành các batch length=10.

Bài 8

Triển khai LSTM 2 tầng với Dropout trong PyTorch. Giải thích ý nghĩa của các tham số num_layers và dropout.

Bài 9

Viết code PyTorch để lấy hidden state cuối cùng từ LSTM và sử dụng nó cho tầng phân loại (classification).

Bài 10

Giải thích tại sao cần pack_padded_sequence trong LSTM? Cho ví dụ với chuỗi có độ dài khác nhau.